Tecnológico De Estudios Superiores De Villa Guerrero

Actividad: Practica 2

MONITOREO DE PROCESOS CON top EN LINUX

Alumna: Arlett Guadalupe Flores Uribe

Carrera: Ingeniería en Sistemas Computacionales

:

Docente: Juan Elias Reynoso

Grupo: 3501

Introducción

En esta práctica se utilizó el comando top en un sistema Linux para observar los procesos activos del sistema en tiempo real. También se revisó información relacionada con el consumo de CPU y memoria, además de utilizar comandos interactivos para ordenar los procesos y gestionar tareas.

Objetivo

Aprender a utilizar el comando top para observar procesos activos, consumo de CPU, memoria y gestionar tareas en tiempo real.

Desarrollo de la práctica

Para comenzar se abrió la terminal

1. Ejecución del comando top

Para comenzar la práctica se ejecutó el siguiente comando:

top

Este comando permite observar los procesos activos del sistema y la información relacionada con su utilización de recursos.

```
user@debian:~$ top

top - 18:50:30 up  1:18,  1 user,  load average: 0.00, 0.07, 0.14
Tasks: 369 total,   2 running, 367 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.1 sy,  0.0 ni, 99.9 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :  13825.1 total,   8814.7 free,   2360.2 used,   3159.1 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.  11464.8 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   4631 user      20   0   10456   6056   3804 R   0.7   0.0   0:00.23 top
```

Resultado: Al ejecutar top se mostró una pantalla que contiene información de los procesos que se están ejecutando en el sistema. La información se actualiza en tiempo real, permitiendo observar el comportamiento de los procesos.

2. Interpretación de las columnas

Durante la ejecución de top se observaron diferentes columnas relacionadas con los procesos:

Columna	Descripción
PID	Identificador del proceso.
USER	Usuario asociado al proceso.
%CPU	Porcentaje de utilización de CPU.
%MEM	Porcentaje de utilización de memoria.

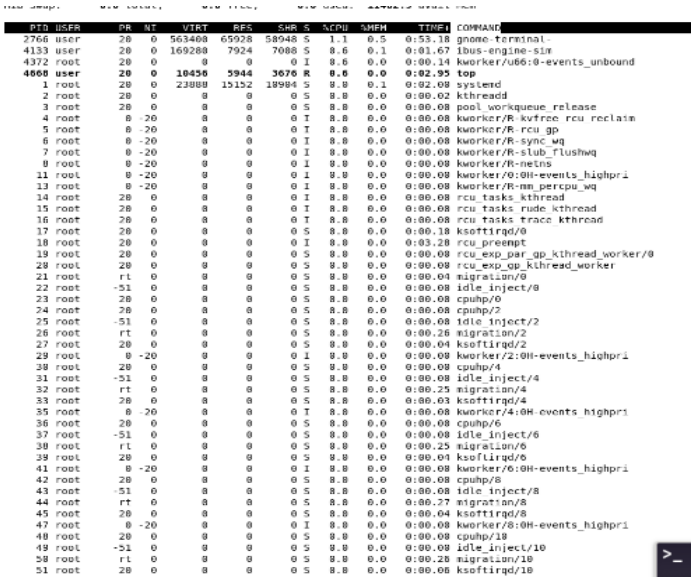
Estas son las columnas que la práctica solicita interpretar.

3. Ordenar procesos por uso de CPU con P

Dentro de la interfaz de top se presionó la tecla:

P

La tecla P permite ordenar los procesos por su uso de CPU de mayor a menor.



```
    PID USER      PR  NT    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2766 user      20   0  563400  65928  58948 S   1.1   0.5   0:53.18 gnome-terminal-
   4133 user      20   0  169288   7924   7088 S   0.6   0.1   0:01.67 ibus-engine-sim
   4372 root      20   0       0      0      0 I   0.6   0.0   0:00.14 kworker/u66:0-events_unbound
   4660 user      20   0   10456   5944   3676 R   0.6   0.0   0:02.95 top
      1 root      20   0   23880  15152  10984 S   0.0   0.1   0:02.08 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.02 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     16 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.18 ksoftirqd/0
     18 root      20   0       0      0      0 I   0.0   0.0   0:03.28 rcu_preempt
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
     21 root      rt   0       0      0      0 S   0.0   0.0   0:00.04 migration/0
     22 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     23 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     24 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     25 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/2
     26 root      rt   0       0      0      0 S   0.0   0.0   0:00.26 migration/2
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.04 ksoftirqd/2
     29 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/2:0H-events_highpri
     30 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/4
     31 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/4
     32 root      rt   0       0      0      0 S   0.0   0.0   0:00.25 migration/4
     33 root      20   0       0      0      0 S   0.0   0.0   0:00.03 ksoftirqd/4
     35 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/4:0H-events_highpri
     36 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/6
     37 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/6
     38 root      rt   0       0      0      0 S   0.0   0.0   0:00.25 migration/6
     39 root      20   0       0      0      0 S   0.0   0.0   0:00.04 ksoftirqd/6
     41 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/6:0H-events_highpri
     42 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/8
     43 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/8
     44 root      rt   0       0      0      0 S   0.0   0.0   0:00.27 migration/8
     45 root      20   0       0      0      0 S   0.0   0.0   0:00.04 ksoftirqd/8
     47 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/8:0H-events_highpri
     48 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/10
     49 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/10
     50 root      rt   0       0      0      0 S   0.0   0.0   0:00.28 migration/10
     51 root      20   0       0      0      0 S   0.0   0.0   0:00.06 ksoftirqd/10
```

Resultado: Después de presionar P, los procesos se ordenaron de acuerdo con el porcentaje de CPU utilizado, mostrando primero los procesos con mayor consumo.

4. Ordenar procesos por uso de memoria con M

Posteriormente, dentro de top, se presionó:

M

Esta tecla permite ordenar los procesos de acuerdo con su uso de memoria.

```
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   2766 user      20   0  563752  66288  50948 S   1.0   0.5   0:55.24 gnome-terminal-
   3324 user      20   0 2517284 158672 115236 S   1.0   1.1   0:09.10 Privileged Cont
   4668 user      20   0   10584   5980   3676 R   1.0   0.0   0:07.42 top
      1 root      20   0   23888  15156  10904 S   0.0   0.1   0:02.02 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.02 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     16 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     17 root      20   0       0      0      0 S   0.0   0.0   0:00.10 ksoftirqd/0
     18 root      20   0       0      0      0 I   0.0   0.0   0:03.54 rcu_preempt
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     20 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
     21 root      rt   0       0      0      0 S   0.0   0.0   0:00.05 migration/0
     22 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     23 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     24 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/2
     25 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/2
     26 root      rt   0       0      0      0 S   0.0   0.0   0:00.26 migration/2
     27 root      20   0       0      0      0 S   0.0   0.0   0:00.05 ksoftirqd/2
     29 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/2:0H-events_highpri
     30 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/4
     31 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/4
     32 root      rt   0       0      0      0 S   0.0   0.0   0:00.26 migration/4
     33 root      20   0       0      0      0 S   0.0   0.0   0:00.03 ksoftirqd/4
     35 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/4:0H-events_highpri
     36 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/6
     37 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/6
     38 root      rt   0       0      0      0 S   0.0   0.0   0:00.26 migration/6
     39 root      20   0       0      0      0 S   0.0   0.0   0:00.04 ksoftirqd/6
     41 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/6:0H-events_highpri
     42 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/8
     43 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/8
     44 root      rt   0       0      0      0 S   0.0   0.0   0:00.27 migration/8
     45 root      20   0       0      0      0 S   0.0   0.0   0:00.04 ksoftirqd/8
     47 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/8:0H-events_highpri
     48 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/10
     49 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/10
     50 root      rt   0       0      0      0 S   0.0   0.0   0:00.26 migration/10
     51 root      20   0       0      0      0 S   0.0   0.0   0:00.06 ksoftirqd/10
     53 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/10:0H-events_highpri
     54 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/12
     55 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle inject/12
```

Resultado: Los procesos fueron reorganizados de acuerdo con el porcentaje de memoria utilizado, permitiendo identificar cuáles procesos tenían un mayor consumo de memoria.

5. Gestionar un proceso con k

Dentro de top también se utilizó la tecla: K

La tecla k permite terminar un proceso y solicita el PID del proceso que se desea gestionar.

```
MiB Swap:      0.0 total,      0.0 free,      0.0 used.  11478.1 avail Mem
PID to signal/kill [default pid = 1867]
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
   1867 user      20   0 6544992 296948 159672 S   9.9   2.1   6:14.98 gnome-shell
   2766 user      20   0  563752  66288  50948 S   2.6   0.5   0:56.19 gnome-terminal-
    347 root     -51   0       0      0      0 S   0.7   0.0   0:20.61 irq/33-ELAN06FA:00
   4818 user      20   0   10456   5892   3648 R   0.7   0.0   0:00.05 top
   1984 user      20   0  460660  12844   7340 S   0.3   0.1   0:15.82 ibus-daemon
   3789 root      20   0       0      0      0 I   0.3   0.0   0:02.51 kworker/u65:1-events_unbound
      1 root      20   0   23888  15156  10904 S   0.0   0.1   0:02.03 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.02 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
     11 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
```

Resultado: Al presionar k, top solicitó el identificador PID del proceso. Esta opción permite seleccionar un proceso para terminarlo mediante su identificador.

6. Salir del comando top

Finalmente, para salir de la interfaz de top, se presionó:

q

La tecla q permite salir del comando top y regresar a la terminal.

```
    41 root       0 -20       0      0      0 I   0.0   0.0
user@debian:~$
```

Resultado: Al presionar q, se cerró la interfaz de monitoreo y se regresó a la línea de comandos de la terminal.

Conclusión

Durante esta práctica se aprendió a utilizar el comando top en Linux para observar los procesos activos y consultar información relacionada con el consumo de CPU y memoria.

También se utilizaron las teclas interactivas P y M para ordenar los procesos según el uso de CPU y memoria. Además, se conoció la función de la tecla k para gestionar un proceso mediante su PID y la tecla q para salir de top.

La práctica permitió comprender de manera práctica cómo monitorear los procesos de un sistema Linux y observar el uso de sus recursos en tiempo real.